DEVOPS • INFRASTRUCTURE AUTOMATION

Ansible + EC2 Ubuntu

A Complete Step-by-Step Study Guide

Setup • Deployment • Security • Networking • Automation

PART 01 Setup

PART 02 Deployment

PART 03 Security

PART 04 Networking

PART 05 Automation

Beginner-friendly • working commands • expected output at every step

Prepared July 2026

What This Guide Covers

This guide takes you from zero to a fully automated EC2 Ubuntu deployment pipeline using Ansible. Every command shown is one you can type yourself, and every step shows the output you should expect to see. Work through the parts in order — each one builds on the setup from the last.

Part	Topic	What You'll Do
Part 1	Setup	Install Ansible, launch EC2 Ubuntu, connect via SSH, build your first inventory
Part 2	Deployment	Install Docker & Nginx, deploy a Node.js app, use handlers and templates
Part 3	Security	SSH hardening, UFW, fail2ban, Ansible Vault for secrets
Part 4	Networking	VPC/subnet/security group basics, multi-host inventories, host groups
Part 5	Automation	Roles, variables, tags, scheduled runs, Jenkins-triggered deployment

BEFORE YOU START You need an AWS account with an EC2 key pair, and a Linux/macOS/WSL terminal for the control node. Everything below assumes Ubuntu 22.04 or 24.04 on the EC2 side.

PART 01

Setup

Get a control node ready, launch an EC2 Ubuntu instance, and connect Ansible to it for the first time.

1.1 What Ansible Actually Does

Ansible is an **agentless** automation tool: it connects to remote machines over plain SSH and runs small Python scripts called **modules**. There is nothing to install on your EC2 instances beforehand — only Python 3, which Ubuntu ships with by default. Your laptop (or a CI server) is the **control node**; the EC2 instances are the **managed nodes**.

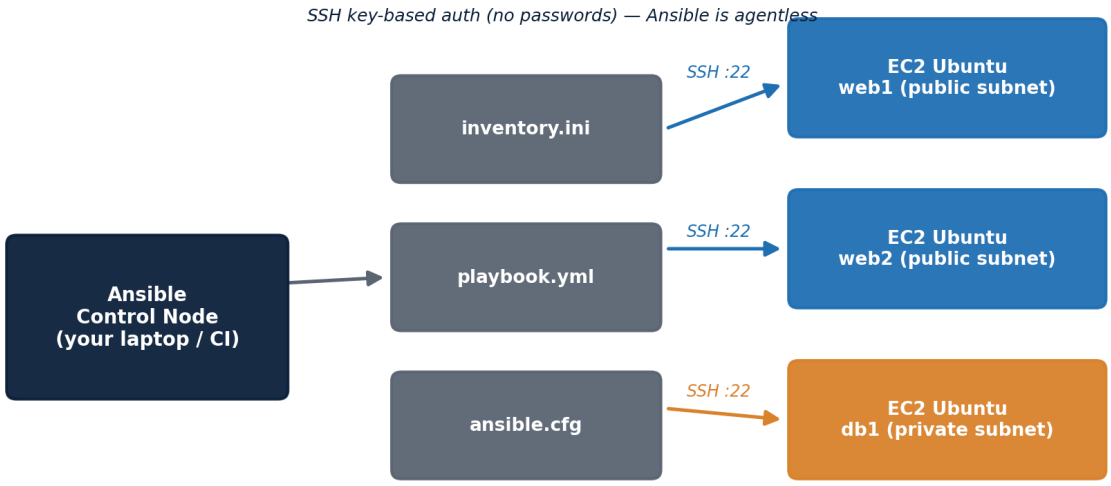


Figure 1: How Ansible reaches EC2 Ubuntu hosts — no software runs on the targets.

Figure 1: How Ansible reaches EC2 Ubuntu hosts — no software runs on the targets.

1.2 Install Ansible on the Control Node

STEP 1 **Install Ansible (Ubuntu/Debian control node)**

● bash

```
$ sudo apt update
$ sudo apt install -y software-properties-common
$ sudo add-apt-repository --yes --update ppa:ansible/ansible
$ sudo apt install -y ansible
```

✓ Expected Output

```
$ ansible --version
ansible [core 2.17.x]
config file = /etc/ansible/ansible.cfg
python version = 3.12.x
```

PLATFORM NOTE On macOS use **brew install ansible**. On Windows, Ansible must run inside WSL2 — it does not run natively on Windows.

STEP 2 Generate an SSH key pair for automation

bash

```
$ ssh-keygen -t ed25519 -f ~/.ssh/ansible_ec2 -C "ansible-control-node"
$ chmod 600 ~/.ssh/ansible_ec2
```

You'll import **ansible_ec2.pub** as the EC2 key pair (or add it as an authorized key after launch).

1.3 Launch the EC2 Ubuntu Instance

STEP 3 Launch from the AWS Console

- AMI: **Ubuntu Server 24.04 LTS** (or 22.04 LTS) — free-tier eligible, HVM, x86_64 or arm64.
- Instance type: **t3.micro** for learning/testing.
- Key pair: select the key you generated, or upload **ansible_ec2.pub** as a new key pair.
- Security group: allow inbound **TCP 22** from your IP for now — Part 4 covers this properly.
- Storage: default 8–20 GB gp3 is fine for a lab instance.

TIP Write down the instance's **public IPv4 address** once it's running — you'll need it for the inventory file next.

1.4 Test SSH Before Involving Ansible

STEP 4 Confirm raw SSH access works

bash

```
$ ssh -i ~/.ssh/ansible_ec2 ubuntu@<EC2_PUBLIC_IP>
```

✓ Expected Output

```
The authenticity of host '203.0.113.42 ...' can't be established.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-1015-aws x86_64)
ubuntu@ip-172-31-20-11:~$
```

If this fails, fix it here first — Ansible will fail the exact same way and hide nothing extra.

1.5 Build the Inventory

STEP 5 Create a project folder and inventory.ini

● bash

```
$ mkdir -p ~/ansible-ec2-lab && cd ~/ansible-ec2-lab
$ nano inventory.ini
```

● inventory.ini

```
[web]
web1 ansible_host=203.0.113.42 ansible_user=ubuntu \
ansible_ssh_private_key_file=~/.ssh/ansible_ec2

[web:vars]
ansible_python_interpreter=/usr/bin/python3
```

STEP 6 Create ansible.cfg so you don't retype flags

● ansible.cfg

```
[defaults]
inventory = inventory.ini
host_key_checking = False
remote_user = ubuntu
private_key_file = ~/.ssh/ansible_ec2
retry_files_enabled = False
```

STEP 7 Ping the host through Ansible

● bash

```
$ ansible web -m ping
```

✓ Expected Output

```
web1 | SUCCESS => {
"ansible_facts": {
"discovered_interpreter_python": "/usr/bin/python3"
},
"changed": false,
"ping": "pong"
}
```

CHECKPOINT SUCCESS + "pong" means the control node, SSH key, and inventory are all correctly wired together. Everything from here on builds on this working connection.

1.6 Your First Playbook

STEP 8 Write and run a minimal playbook

● base-setup.yml

```
---
- hosts: web
become: yes
tasks:
- name: Update apt cache
apt:
update_cache: yes
cache_valid_time: 3600

- name: Install common utilities
apt:
name: [curl, git, unzip, htop]
state: present
```

● bash

```
$ ansible-playbook base-setup.yml
```

✓ Expected Output

```
PLAY [web] ***************************************************

TASK [Gathering Facts] ***************************************
ok: [web1]

TASK [Update apt cache] **************************************
changed: [web1]

TASK [Install common utilities] ******************************
changed: [web1]

PLAY RECAP ***************************************************
web1 : ok=3 changed=2 unreachable=0 failed=0 skipped=0
```

changed=2 means two tasks actually did something. Run the same command again and both will report **ok** instead of **changed** — that's **idempotency**: playbooks are safe to re-run.

PART 02

Deployment

Install Docker and Nginx, and ship a real application to your EC2 instance using idempotent playbooks.



Figure 2: Every ansible-playbook run follows this cycle — re-running is safe (idempotent).

Figure 2: Every ansible-playbook run follows this cycle — re-running is safe.

2.1 Install Docker

STEP 1 **Docker installation playbook**

● **docker-install.yml**

```
---
- hosts: web
become: yes
tasks:
- name: Install prerequisite packages
apt:
name: [ca-certificates, curl, gnupg]
state: present
update_cache: yes

- name: Add Docker's GPG key
apt_key:
url: https://download.docker.com/linux/ubuntu/gpg
state: present

- name: Add Docker apt repository
apt_repository:
repo: "deb https://download.docker.com/linux/ubuntu {{ ansible_distribution_release }}
stable"
state: present

- name: Install Docker Engine
apt:
name: [docker-ce, docker-ce-cli, containerd.io]
state: present
update_cache: yes

- name: Add ubuntu user to docker group
user:
name: ubuntu
groups: docker
append: yes

- name: Ensure Docker is running and enabled
service:
name: docker
state: started
enabled: yes
```

● **bash**

```
$ ansible-playbook docker-install.yml
```

✓ **Expected Output**

```
PLAY RECAP *********************************************************
web1 : ok=7 changed=6 unreachable=0 failed=0 skipped=0
```

GOTCHA The docker-group change only applies to *new* SSH sessions. Ansible reconnects fresh each run, so it's already correct next time — but if you SSH in manually, log out and back in first.

2.2 Install and Configure Nginx

STEP 2 Nginx with a Jinja2 template

Templates let you push dynamic config from variables instead of a static file.

templates/app.conf.j2

```
server {
listen 80;
server_name {{ server_domain }};

location / {
proxy_pass http://127.0.0.1:{{ app_port }};
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
}
}
```

● nginx-deploy.yml

```
---
- hosts: web
become: yes
vars:
server_domain: example.com
app_port: 3000
tasks:
- name: Install Nginx
apt:
name: nginx
state: present

- name: Deploy site config from template
template:
src: templates/app.conf.j2
dest: /etc/nginx/sites-available/app.conf
notify: Reload nginx

- name: Enable the site
file:
src: /etc/nginx/sites-available/app.conf
dest: /etc/nginx/sites-enabled/app.conf
state: link
notify: Reload nginx

- name: Remove default site
file:
path: /etc/nginx/sites-enabled/default
state: absent

handlers:
- name: Reload nginx
service:
name: nginx
state: reloaded
```

CONCEPT A **handler** only runs when a task **notify**'s it, and only once even if several tasks notify it. This avoids reloading Nginx three times in one run for three unrelated config changes.

2.3 Deploy a Node.js Application

STEP 3 **Copy code, install dependencies, run via systemd**

templates/myapp.service.j2

```
[Unit]
Description=My Node App
After=network.target

[Service]
Environment=NODE_ENV=production
ExecStart=/usr/bin/node /opt/myapp/server.js
Restart=always
User=ubuntu
WorkingDirectory=/opt/myapp

[Install]
WantedBy=multi-user.target
```

deploy-app.yml

```
---
- hosts: web
become: yes
vars:
app_port: 3000
tasks:
- name: Install Node.js LTS
apt:
name: nodejs
state: present

- name: Install npm
apt:
name: npm
state: present

- name: Copy application code
synchronize:
src: ./myapp/
dest: /opt/myapp/
delegate_to: "{{ inventory_hostname }}"

- name: Install app dependencies
npm:
path: /opt/myapp
production: yes

- name: Deploy systemd unit
template:
src: templates/myapp.service.j2
dest: /etc/systemd/system/myapp.service
notify: Restart myapp

- name: Enable and start the app
systemd:
name: myapp
enabled: yes
state: started
daemon_reload: yes

handlers:
- name: Restart myapp
systemd:
name: myapp
state: restarted
daemon_reload: yes
```

bash

```
$ ansible-playbook nginx-deploy.yml deploy-app.yml
```

✓ Expected Output

```
PLAY RECAP *****************************************************
web1 : ok=11 changed=9 unreachable=0 failed=0 skipped=0

$ curl -I http://<EC2_PUBLIC_IP>
HTTP/1.1 200 OK
Server: nginx/1.24.0
```

PART 03

Security

Lock down SSH, enable UFW and fail2ban, and manage secrets safely with Ansible Vault.

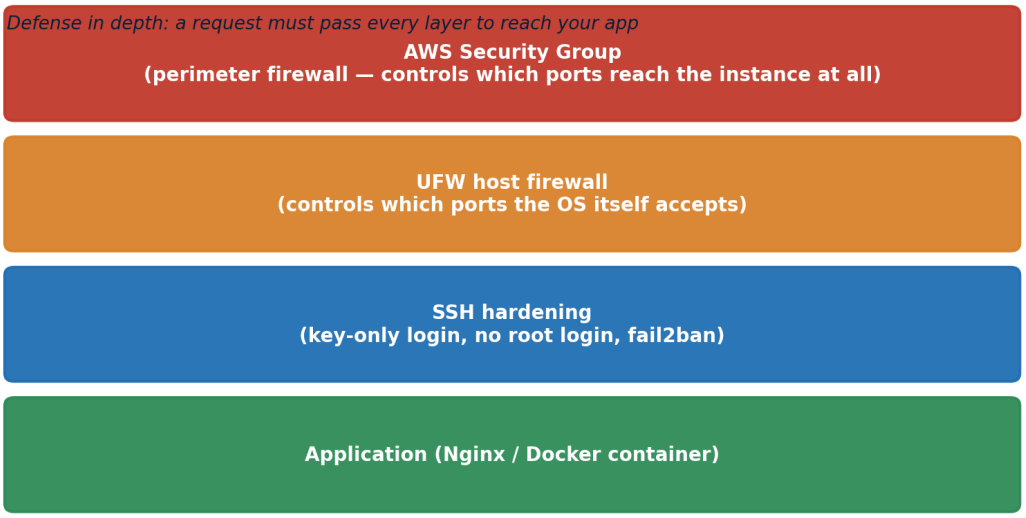


Figure 3: A request must pass every layer before it reaches your app.

3.1 Harden SSH

STEP 1 **Disable password auth and root login**

ssh-harden.yml

```
---
- hosts: web
become: yes
tasks:
- name: Disable password authentication
lineinfile:
path: /etc/ssh/sshd_config
regexp: '^#?PasswordAuthentication'
line: 'PasswordAuthentication no'
notify: Restart sshd

- name: Disable root login
lineinfile:
path: /etc/ssh/sshd_config
regexp: '^#?PermitRootLogin'
line: 'PermitRootLogin no'
notify: Restart sshd

handlers:
- name: Restart sshd
service:
name: ssh
state: restarted
```

WARNING Test in a **second terminal** before closing your first session. If sshd_config is wrong you can lock yourself out; keep one session open as a safety net.

3.2 Firewall with UFW

STEP 2 **Allow only what's needed, then enable**

firewall.yml

```
---
- hosts: web
become: yes
tasks:
- name: Allow SSH
community.general.ufw:
rule: allow
port: '22'
proto: tcp

- name: Allow HTTP
community.general.ufw:
rule: allow
port: '80'
proto: tcp

- name: Allow HTTPS
community.general.ufw:
rule: allow
port: '443'
proto: tcp

- name: Set default deny and enable UFW
community.general.ufw:
state: enabled
policy: deny
```

bash

```
$ ansible-playbook firewall.yml
$ ansible web -m shell -a "sudo ufw status" -b
```

✓ Expected Output

```
web1 | CHANGED | rc=0 >>
Status: active
To Action From
-- ------ ----
22/tcp ALLOW Anywhere
80/tcp ALLOW Anywhere
443/tcp ALLOW Anywhere
```

3.3 fail2ban Against Brute-Force SSH

STEP 3 **Install and enable fail2ban**

● fail2ban.yml

```
---
- hosts: web
become: yes
tasks:
- name: Install fail2ban
apt:
name: fail2ban
state: present

- name: Ensure fail2ban is running and enabled
service:
name: fail2ban
state: started
enabled: yes
```

fail2ban watches auth logs and temporarily bans IPs after repeated failed SSH attempts — a good second layer even with key-only login enforced.

3.4 Secrets with Ansible Vault

STEP 4 **Encrypt sensitive variables at rest**

● bash

```
$ ansible-vault create group_vars/web/vault.yml
```

This opens an editor; type your secrets in normal YAML, then save:

● group_vars/web/vault.yml (plaintext while editing)

```
db_password: "S3cur3-Prod-Password!"
api_key: "sk-live-xxxxxxxxxxxxxxx"
```

● bash

```
$ ansible-playbook deploy-app.yml --ask-vault-pass
# or, for CI:
$ ansible-playbook deploy-app.yml --vault-password-file ~/.vault_pass.txt
```

✓ Expected Output

```
$ cat group_vars/web/vault.yml
$ANSIBLE_VAULT;1.1;AES256
66386439653766326135633534366263373661326462633461...
(file is now unreadable without the vault password)
```

BEST PRACTICE Never commit **~/.vault_pass.txt** or the unencrypted secrets to Git. Commit the encrypted **vault.yml** file — that part is safe to store in version control.

PART 04

Networking

Understand VPCs, subnets and security groups, and manage multi-host inventories with groups.

4.1 VPC, Subnets, and Security Groups — the Basics

Concept	What It Controls	Typical Setting for This Lab
VPC	The isolated network your EC2 instances live in	Default VPC is fine for learning
Subnet	Which Availability Zone, and public vs private routing	Public subnet for web tier
Security Group	Stateful firewall attached to the instance (AWS layer)	Allow 22, 80, 443 only
Network ACL	Stateless firewall attached to the subnet	Leave as default unless needed
Route Table	Where subnet traffic is sent (e.g. to an Internet Gateway)	Default route → IGW for public

CONCEPT A Security Group is **stateful**: if you allow inbound port 22, the reply traffic is automatically allowed out. You almost never need matching outbound rules for normal SSH/HTTP traffic.

4.2 Restricting the Security Group Properly

In Part 1 you allowed SSH from "my IP" for convenience. For anything beyond a personal lab:

- Restrict port 22 to your office/VPN CIDR block, not 0.0.0.0/0.
- Only open 80/443 to the world — the app port (e.g. 3000) should never be internet-facing directly.
- Put a database instance in a **private subnet** with a security group that only allows traffic from the web tier's security group, not from any IP.

concept, not Ansible

```
# Referencing another security group instead of a CIDR (AWS console or Terraform),
# conceptually:
DB security group inbound rule:
Type: MySQL/Aurora (3306)
Source: sg-0123456789abcdef0 (the WEB security group, not 0.0.0.0/0)
```

4.3 Multi-Host Inventory with Groups

STEP 1 **Group hosts by role**

● inventory.ini

```
[web]
web1 ansible_host=203.0.113.42
web2 ansible_host=203.0.113.51

[db]
db1 ansible_host=10.0.2.15 # private subnet, reached via bastion/VPN

[web:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/ansible_ec2

[all:children]
web
db
```

Now a playbook can target **-l web**, **-l db**, or **-l all**, and **group_vars/web.yml** / **group_vars/db.yml** apply variables automatically to only that group.

STEP 2 Reach a private-subnet host through a bastion

● inventory.ini (bastion jump)

```
[db]
db1 ansible_host=10.0.2.15 ansible_ssh_common_args='-o ProxyCommand="ssh -W %h:%p -q
ubuntu@203.0.113.42 -i ~/.ssh/ansible_ec2"'
```

CONCEPT The **bastion** (or a NAT/jump box) is the only host in the private subnet's security group allowed to SSH in — this is how you manage database servers without exposing them publicly.

4.4 Dynamic Inventory (Optional, Scales Better)

Instead of hand-editing IPs, the **amazon.aws.aws_ec2** inventory plugin queries AWS directly by tag, so new instances appear automatically.

● aws_ec2.yml

```
plugin: amazon.aws.aws_ec2
regions:
- ap-south-1
filters:
tag:Role: web
keyed_groups:
- key: tags.Role
prefix: role
```

● bash

```
$ pip install boto3 botocore --break-system-packages
$ ansible-galaxy collection install amazon.aws
$ ansible-inventory -i aws_ec2.yml --graph
```

✓ Expected Output

```
@all:
|--@role_web:
| |--ec2-web1
| |--ec2-web2
|--@ungrouped:
```

PART 05

Automation

Package your work into roles, variables and tags, and trigger deployments automatically from Jenkins.

5.1 Refactor into Roles

Once a playbook grows past ~50 lines, split it into a **role** — a standard folder structure Ansible auto-discovers, which is what real production repos use.

bash

```
$ ansible-galaxy init roles/webserver
```

role layout

```
roles/webserver/
tasks/main.yml # the actual steps
handlers/main.yml # notify targets
templates/ # .j2 files
defaults/main.yml # default variable values (lowest priority)
vars/main.yml # role-specific variables
files/ # static files to copy as-is
```

site.yml (now just glue)

```
---
- hosts: web
become: yes
roles:
- webserver
- docker
- firewall
```

CONCEPT This is the same content as Parts 1–3, reorganized. Roles don't add new capability — they add structure so ten playbooks become one readable site.yml.

5.2 Variables and Precedence

Source	Where It Lives	Overrides
Role defaults	roles/x/defaults/main.yml	Lowest priority — safe fallback values
group_vars	group_vars/web.yml	Overrides role defaults for that group
host_vars	host_vars/web1.yml	Overrides group_vars for one host
-e on the CLI	ansible-playbook site.yml -e env=prod	Highest priority — wins every time

5.3 Tags for Partial Runs

STEP 1 Tag tasks so you can run a slice of the playbook

excerpt

```
- name: Deploy site config
template:
src: app.conf.j2
dest: /etc/nginx/sites-available/app.conf
tags: [nginx, config]
```

bash

```
$ ansible-playbook site.yml --tags nginx
$ ansible-playbook site.yml --skip-tags firewall
```

5.4 Scheduled and Triggered Automation

STEP 2 Run Ansible on a schedule with cron

crontab

```
# On the control node's crontab (crontab -e)
# Re-apply configuration every night at 2am, log the result
0 2 * * * cd /home/hari/ansible-ec2-lab && ansible-playbook site.yml >>
/var/log/ansible-nightly.log 2>&1
```

STEP 3 Trigger from Jenkins after a Git push

A typical pipeline: GitHub push → Jenkins job triggers → Jenkins checks out the repo → runs the playbook against production.

● Jenkinsfile

```
pipeline {
agent any
stages {
stage('Checkout') {
steps { git branch: 'main', url: 'https://github.com/you/myapp.git' }
}
stage('Deploy') {
steps {
sh '''
ansible-playbook -i inventory.ini site.yml \
--vault-password-file /var/jenkins_home/.vault_pass.txt
'''
}
}
}
}
```

CONNECTION This is the same GitHub-to-EC2 flow from your Jenkins CI/CD guide — Ansible simply replaces the raw shell/SCP steps with idempotent, re-runnable playbooks.

5.5 Quick Command Reference

Command	Purpose
ansible web -m ping	Test connectivity to the web group
ansible-playbook site.yml --check	Dry run — shows changes without applying them
ansible-playbook site.yml --diff	Show the actual file/line differences being made
ansible-playbook site.yml -l web1	Limit the run to a single host
ansible-playbook site.yml --tags nginx	Run only tasks tagged 'nginx'
ansible-vault edit vault.yml	Edit an encrypted secrets file in place
ansible-inventory --graph	Visualize the current inventory structure

End-to-End Checklist

- Control node has Ansible installed and an SSH key generated (Part 1).
- EC2 Ubuntu instance launched, reachable, and pinging successfully through Ansible (Part 1).
- Docker/Nginx/app deployed idempotently with handlers and templates (Part 2).
- SSH hardened, UFW enabled, fail2ban running, secrets in Vault (Part 3).
- Security groups scoped tightly; multi-host inventory with groups in place (Part 4).
- Playbooks refactored into roles, variables layered correctly, deployment automated via Jenkins/cron (Part 5).